

HTTP Request Smuggling & HTTP Response Splitting

These two vulnerability classes both stem from **disagreement about where one HTTP message ends and the next begins**, but they operate on opposite ends of the request/response chain — worth locking in the distinction immediately: **request smuggling** exploits a front-end/back-end disagreement over where a *request* ends, letting an attacker "smuggle" a hidden second request that gets processed against a different, unsuspecting victim; **response splitting** exploits unsanitized input reflected into a *response header*, letting an attacker inject characters that trick something downstream into treating one response as two. One weaponizes ambiguity on the way *in* to the server; the other weaponizes injection on the way *out*. Both connect directly back to your Cache Poisoning parser-discrepancy discussion and your XSS/Open Redirect delivery concepts from earlier guides.

1. How Chained HTTP Infrastructure Works — The Foundation

Almost no modern application talks to users directly. A front-end (load balancer, CDN, reverse proxy) terminates the connection and forwards the request to a back-end (application server) over a **reused, persistent TCP connection** — often multiplexing many different users' requests over the same pipe to the back-end for performance.

1. Client sends Request A over Connection 1 to the Front-end
2. Front-end forwards Request A to the Back-end over a POOLED connection (shared/reused across many different clients)
3. Back-end processes Request A, sends response back
4. Front-end reuses that SAME pooled connection for Request B, Request C, etc. from OTHER, unrelated clients

The message boundary — the entire concept both vulnerabilities exploit:

HTTP/1.1 has **two different ways** to tell a recipient how long a message body is — **Content-Length** (a byte count) and **Transfer-Encoding: chunked** (a chunk-by-chunk sentinel-terminated stream). If the front-end and back-end **disagree** about which of these to trust, or disagree about how to parse a malformed version of

either, they will disagree about **where the request/response actually ends** — and everything after that point gets treated as the start of the *next* message.

```
Front-end's view of where the message ends    → Point X
Back-end's view of where the SAME message ends → Point Y (≠ X)
Whatever bytes sit between X and Y           → get interpreted as
                                              the START of the
                                              NEXT message on that
                                              connection – attacker-
                                              controlled, injected
                                              into someone else's context
```

This single fact — two parsers disagreeing on message length — is the entire root cause of HTTP request smuggling. Response splitting is this same root idea applied one layer up: not disagreement between two *servers*, but injection of raw CRLF sequences into a single response so that a downstream cache/browser is tricked into seeing two responses instead of one.

PART 1: HTTP Request Smuggling

2. The Core Attack Model

```
Phase 1: Craft a single HTTP request containing a SMUGGLED
         second request hidden inside it, using a Content-Length/
         Transfer-Encoding discrepancy the front-end and back-end
         parse differently

Phase 2: The front-end forwards what IT believes is one request.
         The back-end processes what IT believes is one request –
         and then treats the smuggled leftover bytes as the
         BEGINNING of the next request on that connection, which
         belongs to a DIFFERENT, unsuspecting victim user

Phase 3: The victim's genuine next request gets APPENDED to the
         attacker's smuggled prefix – meaning the back-end
         processes a request that is part attacker-controlled,
         part victim-controlled, in the VICTIM's session context
```

Why this is so severe compared to a "normal" injection bug: the attacker never touches the victim directly at all — no phishing link, no XSS payload, nothing rendered in the victim's browser. The attacker corrupts a **shared, pooled connection** at the infrastructure level, and the very next real user unlucky enough to reuse that connection has their request silently hijacked,

response redirected, or session compromised — entirely server-side, entirely invisible to the victim.

3. The Three Classic Discrepancy Types

```
CL.TE → Front-end trusts CONTENT-LENGTH
      Back-end trusts TRANSFER-ENCODING
      (front-end forwards the whole body as one request; back-end
      reads only up to where its chunked parsing says the body ends,
      then treats the remainder as a new pipelined request)

TE.CL → Front-end trusts TRANSFER-ENCODING
      Back-end trusts CONTENT-LENGTH
      (inverse of the above – back-end reads a short, fixed-length
      body and treats the rest of the "chunked" data as a new request)

TE.TE → BOTH sides nominally support Transfer-Encoding, but one
      of them can be tricked into NOT recognizing the header at
      all via obfuscation – whitespace, casing, duplicate headers,
      invalid chunk sizes – causing it to silently fall back to
      Content-Length while the other still parses it as chunked
```

4. CL.TE — Basic Smuggling Request

This is the canonical, simplest form of the attack:

```
POST / HTTP/1.1
Host: vulnerable-website.com
Content-Length: 13
Transfer-Encoding: chunked

0

SMUGGLED
```

The front-end trusts `Content-Length: 13` and forwards exactly that many bytes as the request body. The back-end trusts `Transfer-Encoding: chunked`, reads the `0\r\n\r\n` as a **terminating chunk of size zero**, considers the request body finished right there — and treats the literal text `SMUGGLED` sitting after it as the **start of the next pipelined request** on that connection, which then gets glued onto whatever real request the next actual user sends.

5. TE.CL — The Inverse Attack

```
POST / HTTP/1.1
Host: vulnerable-website.com
Content-Length: 3
Transfer-Encoding: chunked

8
SMUGGLED
0
```

Here the front-end processes this as chunked (reads chunk size `8`, reads 8 bytes of `SMUGGLED`, reads terminating `0` chunk — the whole thing is one request). The back-end instead trusts `Content-Length: 3`, reads only the first 3 bytes (`8\r\n`) as the complete body, and treats everything after that — the rest of the chunked payload — as the start of a new smuggled request.

6. TE.TE — Obfuscating the Transfer-Encoding Header

```
Transfer-Encoding: chunked
Transfer-Encoding: cow

Transfer-Encoding:[tab]chunked

Transfer-Encoding: chunked
```

If either the front-end or the back-end can be made to **not recognize** a malformed/duplicated/obfuscated `Transfer-Encoding` header while the other one still does, you've re-derived a CL.TE or TE.CL discrepancy through obfuscation rather than through the two servers genuinely disagreeing on protocol support — same underlying class of bug as Sections 4-5, just via header-parsing evasion instead of outright dual-header ambiguity.

7. Confirming Smuggling — Timing-Based Detection

Since the exploit's actual effect only becomes visible on the *next* request on the connection, the safest confirmation technique doesn't touch another user's traffic at all:

1. Send a CL.TE probe where the smuggled portion is a request with NO trailing data the back-end is still expecting (e.g. smuggle a request whose Content-Length claims a body that never arrives)
2. If the back-end is left HANGING, waiting for bytes that will never come, the RESPONSE to your own probe request is delayed

3. A consistent, reproducible time delay confirms the back-end parsed the request differently than the front-end did – a working smuggling discrepancy, with zero risk to real traffic

8. Exploiting Confirmed Smuggling — Real-World Impact

Bypassing front-end security controls → smuggle a request that the front-end WAF would normally block, disguised inside a request the front-end considers benign

Capturing other users' requests → smuggle a request that stores the NEXT user's raw request inside a resource you can retrieve (e.g. into a comment field, a logged parameter)

Session hijacking / response takeover → cause the victim's subsequent request to receive the ATTACKER's response instead of their own, or vice versa

Reflected XSS delivery without user interaction → smuggle a request that causes the FRONT-END to cache or reflect an injected payload for the next visitor – this directly converges with your Cache Poisoning guide's delivery model

Web cache poisoning via smuggling → smuggle a request whose response gets cached under a shared/normal-looking cache key, turning smuggling into a poisoning delivery mechanism

9. HTTP/2 Downgrading — The Modern Variant

Many front-ends speak HTTP/2 to clients but downgrade to HTTP/1.1 to talk to the back-end. Since HTTP/2 uses explicit, length-prefixed binary framing (no `Content-Length` / `Transfer-Encoding` ambiguity is even possible in native HTTP/2), the entire vulnerability class re-emerges specifically at the **downgrade step**:

- Injecting a request via a crafted HTTP/2 pseudo-header (e.g. a header value containing an embedded CRLF or an oversized/duplicate Content-Length) that gets translated into an ambiguous HTTP/1.1 request during downgrade
- CRLF injection into an HTTP/2 header VALUE – normally impossible for a client to trigger directly – becomes possible if the front-end fails to strictly validate header values before generating the downgraded HTTP/1.1 request text

This is exactly why your Prevention section (13) calls out preferring native HTTP/2 end-to-end as a structural fix, not just a patch.

PART 2: HTTP Response Splitting

10. The Core Concept — Injection, Not Disagreement

Where smuggling exploits two parsers disagreeing about a request boundary, splitting exploits **unsanitized attacker input landing inside a response header**, allowing the attacker to inject raw **CRLF** (`\r\n`) sequences and manufacture a second, fully attacker-controlled response out of what should have been a single header value.

1. Application reflects UNSANITIZED user input into a response header (a redirect Location header, a Set-Cookie value, a custom header built from a query parameter)
2. Attacker injects CRLF characters plus fabricated header/body content into that input
3. The recipient (browser, or more dangerously, an intermediate CACHE) parses the injected CRLFs as genuine header terminators, and sees what looks like TWO separate HTTP responses instead of the one the server actually sent
4. The attacker's fabricated "second response" is now processed independently – cached, rendered, or acted on as if it were legitimate server output

11. Classic Exploitation Pattern

```
Vulnerable reflection point:
  Location: /redirect?url=USER_INPUT

Attacker-crafted input:
  USER_INPUT = /%0d%0aSet-Cookie:%20session=attacker-value%0d%0a
               %0d%0a<html><script>alert(document.cookie)</script></html>
```

The URL-encoded `%0d%0a` sequences decode to literal **CRLF** once the server builds the header. If the server doesn't strip or reject these characters when constructing the **Location** header, the resulting raw response stream now contains attacker-injected header lines and even a fabricated response body — smuggled entirely inside what was meant to be a single header value.

12. Why This Escalates Beyond the Browser — Cache Interaction

Response splitting on its own, against a modern browser, is often heavily mitigated (most browsers reject raw CRLF in redirected requests and modern frameworks encode header values automatically). Its real danger today is almost always **in combination with an intermediate cache**:

- If a CACHING PROXY sits between the vulnerable server and the end user, and the split, fabricated "second response" gets interpreted and STORED by that cache under a normal-looking key
- Every subsequent visitor requesting that same resource now receives the attacker's fabricated response instead of the genuine one

This is precisely the **Web Cache Poisoning** delivery model from your earlier guide — response splitting is one specific *technique* for achieving the same "malicious content injected into the cache" outcome that Section 4 of that guide reached via unkeyed headers instead of CRLF injection. Same destination, different on-ramp.

13. Testing Methodology for Response Splitting

1. Identify every point where user-controllable input is reflected into a RESPONSE HEADER (redirects, custom headers, Set-Cookie values built from parameters, framework "flash message" headers)
2. Inject raw CRLF sequences (both literal and URL/double-encoded: %0d%0a, %250d%250a) followed by a fabricated header line
3. Check whether the fabricated header appears as a GENUINE, separate header in the raw response (use a raw TCP/Burp Repeater view, not a browser, since browsers may silently normalize this)
4. If a fabricated header lands, attempt injecting a full second response (headers + blank line + body) to confirm true splitting, not just single-header injection
5. Test through any intermediate CACHE specifically – request the same crafted URL from a second, clean session afterward to check whether the fabricated response got stored and is now served to other users

14. Prevention — HTTP Request Smuggling

- Use HTTP/2 END-TO-END across every layer of the chain (front-end to back-end) - eliminates Content-Length/Transfer-Encoding ambiguity entirely, since HTTP/2 framing has no such ambiguity to exploit
- If HTTP/1.1 must be used internally, ensure the front-end and back-end are from the SAME vendor/implementation, or explicitly tested for identical parsing behavior on malformed/duplicate Content-Length and Transfer-Encoding headers
- Disable connection reuse/pipelining between front-end and back-end where feasible - removes the shared-connection precondition smuggling depends on entirely (a performance/security trade-off)
- Reject, don't normalize, ambiguous requests - a request containing

- BOTH Content-Length and Transfer-Encoding, or a malformed/duplicated version of either, should be dropped outright rather than each layer independently guessing which one to honor
- Normalize the request identically at every hop before forwarding, rather than trying to selectively patch just the discrepancies you've already found exploitable

15. Prevention — HTTP Response Splitting

- NEVER build response headers via raw string concatenation of user input - use a framework/library's structured header-setting API, which encodes or rejects CRLF automatically
- Explicitly strip or reject CR (%0d) and LF (%0a) characters - encoded or literal - from any value destined for a response header
- Set explicit Cache-Control: private / no-store on any response containing reflected, user-influenced content, so even a successful split can't be leveraged into a cache-wide compromise
- Validate redirect targets (Location header values) against an allowlist rather than reflecting attacker-supplied URLs directly
- Keep intermediate caches and CDNs strict about what constitutes a valid single response - reject malformed responses with unexpected extra header blocks rather than attempting to parse and serve whatever follows

16. Practical Lab Example

PortSwigger's Web Security Academy has a dedicated lab set for "HTTP Request Smuggling", covering CL.TE, TE.CL, and TE.TE individually, plus dedicated HTTP/2-downgrading labs reflecting James Kettle's more recent research on the topic. Response Splitting is covered more thinly as a standalone modern lab set, since it's increasingly folded into general header-injection and cache poisoning material rather than treated as fully separate.

Suggested practice order:

1. Request smuggling: CL.TE basic lab (Section 4's exact pattern)
 - use Burp's built-in HTTP Request Smuggler extension rather than hand-crafting byte counts at first, to build intuition before doing it manually
2. Request smuggling: TE.CL and TE.TE labs (Sections 5-6) once the basic mental model clicks
3. Request smuggling: HTTP/2 downgrading lab (Section 9) - the clearest illustration of how a "solved" HTTP/1.1 problem re-emerges at a protocol translation boundary
4. Response splitting: manual CRLF injection into a redirect parameter (Section 11), then chain it against a caching lab to see the convergence with cache poisoning first-hand

Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

Framework	Mapping
CWE	CWE-444 (HTTP Request/Response Smuggling - the shared parser-discrepancy family, same entry referenced in your Cache Poisoning guide), CWE-113 (HTTP Response Splitting / CRLF Injection specifically)
OWASP Top 10:2025	A05:2025 Injection (both classes are fundamentally injection into a protocol-parsing boundary rather than an application logic layer) - smuggling's downstream effects also touch A02:2025 Security Misconfiguration when it stems from mismatched front-end/back-end server software
CVSS	Smuggling frequently reaches Critical when it enables session hijacking or WAF/auth bypass at scale with zero victim interaction (UI:N, PR:N); response splitting severity is usually High and scales up sharply when a cache is in the delivery path, mirroring the poisoning severity model
Cyber Kill Chain	Delivery (smuggling turns the shared connection itself into the delivery mechanism, splitting turns a single header into a delivery mechanism) → Exploitation (hijacked session/response reaching the victim)
MITRE ATT&CK	T1071.001 (Web Protocols, as the abused channel) for both; smuggling's session-hijacking outcomes overlap with T1550 (Use Alternate Authentication Material) once a victim's session is captured

Quick Reference Cheat Sheet

REQUEST SMUGGLING (exploit disagreement over where a REQUEST ends):

1. Identify a front-end + back-end pair in the chain
2. Send CL.TE or TE.CL probe with a deliberately unresolved smuggled remainder (Section 7's timing technique)
3. Confirm via response delay, or via smuggling a request into a storable location (comment field, log) and retrieving it
4. Weaponize: hijack the next user's request/response, bypass a front-end WAF, or chain into cache poisoning

RESPONSE SPLITTING (exploit injection into a RESPONSE header):

1. Find user input reflected into a response header (redirects, custom headers, Set-Cookie)
2. Inject CRLF (literal or encoded: %0d%0a) plus a fabricated header/body
3. Confirm via raw TCP/Repeater view that a second, fabricated

response is recognized by the recipient

4. Weaponize: chain into an intermediate cache so the fabricated response is stored and served to other users

Root cause shared by both: a boundary-parsing ambiguity - smuggling between TWO SERVERS parsing the same REQUEST differently, splitting within ONE SERVER's output being reinterpreted as TWO RESPONSES by whatever parses it downstream
